

UNMATCHED

BASKERVILLE MANOR

ADVANCED PROGRAMMING PROJECT REPORT

GRAPHICAL VERSION

C++ • Object-Oriented Programming • SFML • JSON Serialization



Instructor: Dr. Yousef Sanati

Created by:

Sonia Sate — 40412358022

Zahra Akbari — 40412358006

In memory of those whose lives were silenced, but whose voices could never be silenced.

Table of Contents

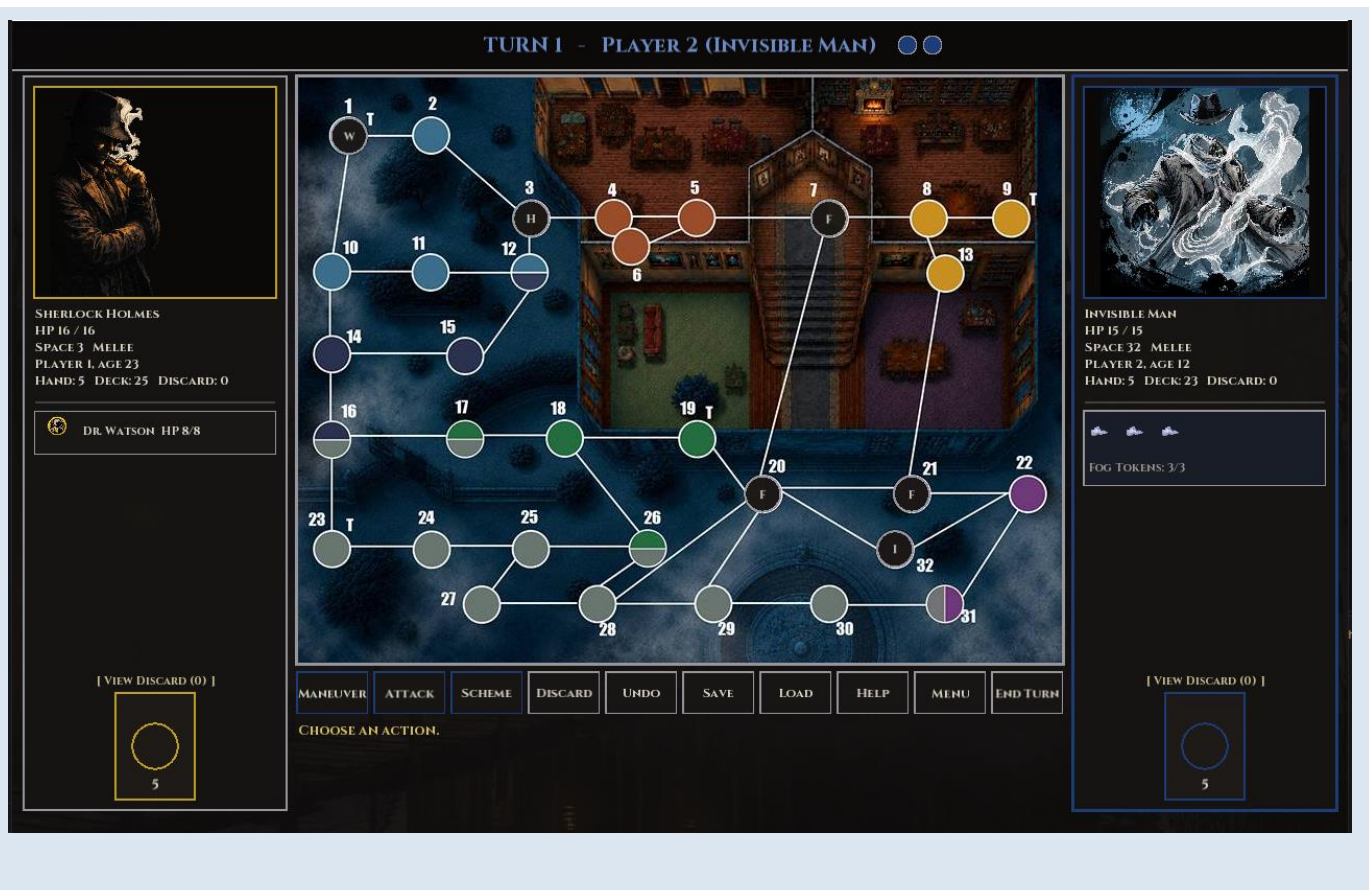
1. Project Overview
2. Project Structure
3. Core Game Model
4. Board and Movement
5. GameController and Actions
6. Character-Specific Mechanics
7. Graphical Interface and Game Loop
8. GameScreen State Management
9. Resource Management
10. Save, Load and Undo
11. Exception Handling and OOP
12. Challenges
13. Repository
14. Resources

1. Project Overview

Phase 2 of this project focuses on the graphical version of Unmatched. The core game logic from Phase 1 is preserved, while this phase adds an SFML interface, game menus and setup screens, the Invisible Man with Fog Token mechanics, save/load support, and an Undo system.

1. معرفی کلی پروژه

فاز دوم این پروژه بر نسخه گرافیکی بازی Unmatched تمرکز دارد. منطق اصلی بازی که در فاز اول پیاده‌سازی شده بدون تغییر حفظ شده است و در این فاز رابط گرافیکی با SFML، منوها و مراحل شروع بازی، شخصیت Invisible Man و مکانیزم Fog Token، قابلیت Save/Load و سیستم Undo به پروژه اضافه شده‌اند.

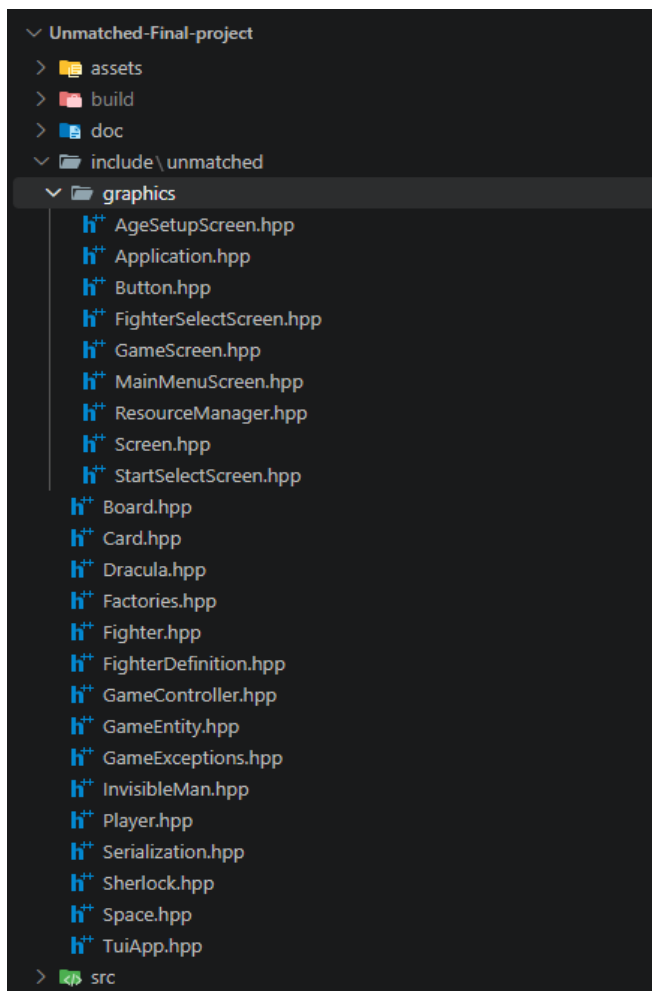


2. Project Structure

The project is divided into small classes with separate responsibilities. Important core classes include Board, Space, Player, Card, Fighter and GameController. Character classes such as Dracula, Sherlock and InvisibleMan inherit from Fighter. The graphical part contains Application, Screen, Button, ResourceManager and several screen classes.

2. ساختار پروژه

پروژه به کلاس‌های کوچکتر با مسئولیت‌های مشخص تقسیم شده است. مهم‌ترین کلاس‌های اصلی شامل Board، Space، Player، Card، Fighter و GameController هستند. کلاس‌های Dracula، Sherlock و InvisibleMan از Fighter ارث‌بری می‌کنند. بخش گرافیکی نیز شامل Application، Screen، Button، ResourceManager و Screen‌های مختلف است.



3. Core Game Model

GameEntity provides common identity information for game objects. Fighter stores the current state of a fighter, while FighterDefinition stores fixed character information such as maximum health, movement and attack range. Card stores card type, attack/defense values, boost, timing and effect. Player owns fighters, deck, hand and discard pile.

3. هسته ی بازی

کلاس GameEntity اطلاعات عمومی موجودیت‌ها را نگهداری می‌کند. Fighter وضعیت فعلی مبارز و FighterDefinition اطلاعات ثابت شخصیت مانند حداکثر جان، مقدار حرکت و نوع حمله را نگهداری می‌کند. Card شامل نوع کارت، مقدار حمله و دفاع، Boost، Timing و Effect است. Player نیز مبارزان، Deck، Hand و Discard Pile را مدیریت می‌کند.

4. Board and Movement

In Phase 2, the original Baskerville Manor board rules remain unchanged. The main movement extension is related to the Invisible Man and his Fog Tokens. Fog spaces can act as connected movement points for the Invisible Man, while normal adjacency and zone rules are still preserved.

4. نقشه و حرکت در فاز دوم

در فاز دوم قوانین اصلی نقشه Baskerville Manor تغییر نکرده‌اند. مهم‌ترین توسعه در بخش حرکت مربوط به Invisible Man و Fog Token های اوست. خانه‌های دارای Fog برای حرکت Invisible Man می‌توانند نقش مسیر متصل را داشته باشند، در حالی که قوانین اصلی Adjacency و Zone همچنان حفظ می‌شوند.

5. GameController and Actions

GameController was extended to support the new Phase 2 interactions. It handles Fog Token movement, Vanish placement and multi-step card choices. Pending states are used when a card needs another selection from the player, allowing the graphical interface to continue without blocking the game loop.

5. تغییرات GameController و اکشن‌ها

GameController برای تعاملات جدید فاز دوم توسعه داده شده است. حرکت Fog Token ها، قرارگیری دوباره شخصیت بعد از Vanish و انتخاب‌های چندمرحله‌ای کارت‌ها در این بخش مدیریت می‌شوند. برای کارت‌هایی که به انتخاب بعدی بازیکن نیاز دارند از Pending State استفاده شده تا رابط گرافیکی بدون متوقف کردن Game Loop ادامه پیدا کند.

6. Character-Specific Mechanics

The new playable character in Phase 2 is the Invisible Man. He has 15 health, 2 movement and a melee attack. Instead of a sidekick, he starts with three Fog Tokens. When defending on a Fog space, his defense value is increased by one. The Fog Tokens are also used by several character cards for movement, damage and repositioning.

- Three Fog Tokens are placed at the beginning of the game.
- Fog Tokens are moved by card effects, not by the normal Maneuver action.
- Phase 2 implements the Invisible Man deck and its multi-step effects.

6. مکانیزم‌های شخصیت Invisible Man

شخصیت جدید فاز دوم Invisible Man است. او ۱۵ واحد جان، ۲ واحد حرکت و حمله Melee دارد. به جای Sidekick، بازی را با سه Fog Token آغاز می‌کند. هنگام دفاع روی خانه‌ای که Fog Token دارد، یک واحد به مقدار دفاع او اضافه می‌شود. همچنین تعدادی از کارت‌های این شخصیت برای حرکت، آسیب یا جابه‌جایی از Fog Token ها استفاده می‌کنند.

- سه Fog Token در ابتدای بازی روی نقشه قرار می‌گیرند.
- Fog Token ها با اثر کارت‌ها جابه‌جا می‌شوند و بخشی از Maneuver عادی نیستند.
- Deck شخصیت Invisible Man و Effect های چند مرحله‌ای کارت‌های او در فاز دوم پیدامسازي شده‌اند.

```
1  #pragma once
2
3  #include "unmatched/Fighter.hpp"
4  #include <vector>
5
6  namespace unmatched {
7
8  class InvisibleMan : public Fighter {
9  public:
10     InvisibleMan();
11
12     std::string getSpecialAbility() const override;
13     bool canPlayCard(const Card& card) const override;
14     std::vector<Card> createDeck() const override;
15     std::vector<std::unique_ptr<Fighter>> createSidekicks() const override;
16
17     std::vector<int>& getFogTokens();
18     const std::vector<int>& getFogTokens() const;
19     void placeFogToken(int index, int spaceId);
20     void moveFogToken(int index, int spaceId);
21     void placeFogTokens(const std::vector<int>& spaces);
22     bool isOnFog(int spaceId) const;
23     bool hasFogAt(int spaceId) const;
24     std::vector<int> getFogSpaces() const;
25     void resetFogTokens();
26     int getDefenseBonus() const { return 1; }
27     void onTurnStart() override;
28     int getStartTurnSpace() const { return startTurnSpace_; }
29
30 private:
31     std::vector<int> fogTokens_;
32     int startTurnSpace_ = -1;
33 };
34
35 } // namespace unmatched
36
```

7. Graphical Interface and Game Loop

The Phase 2 interface is implemented with SFML. Application owns the 1280×800 window and runs the event-update-render loop at 60 FPS. All pages inherit from Screen and implement handleEvent(), update() and render().

- Main Menu: Start, Load Game and Exit.

- Age Setup: receives both ages and determines the younger player.
- Fighter Selection and Start Selection: complete the setup before gameplay.
- Game Screen: displays the board, fighters, cards, actions and status information.

7. رابط گرافیکی و Game Loop

رابط گرافیکی فاز دوم با SFML پیاده‌سازی شده است. کلاس Application پنجره 1280×800 را مدیریت می‌کند و Game Loop شامل Event ، Update و Render را با محدودیت 60 FPS اجرا می‌کند. تمام صفحات نیز از Screen ارث‌بری کرده و متدهای handleEvent() ، update() و render() را پیاده‌سازی می‌کنند.

- Main Menu شامل Start ، Help و Load Game و Exit است.
- Age Setup سن دو بازیکن را دریافت کرده و بازیکن جوان‌تر را مشخص می‌کند.
- Fighter Selection و Start Selection مراحل قبل از شروع بازی را تکمیل می‌کنند.
- Game Screen نقشه، مبارزان، کارت‌ها، اکشن‌ها و وضعیت بازی را نمایش می‌دهد.

8. GameScreen State Management

GameScreen uses an internal Mode enum for multi-step graphical input. Separate modes are used for Maneuver, Attack, Scheme, Fog Token selection, Vanish placement, card-specific choices and Game Over. This keeps mouse input consistent with the current action.

8. مدیریت حالت‌های GameScreen

GameScreen برای ورودی‌های چندمرحله‌ای از enum داخلی Mode استفاده می‌کند. برای Maneuver، Attack، Scheme، انتخاب Fog Token، محل قرارگیری Vanish، انتخاب‌های مخصوص کارت‌ها و Game Over حالت‌های جداگانه وجود دارد. به این ترتیب رفتار Mouse همیشه با اکشن فعلی هماهنگ می‌ماند.

```
enum class Mode {
    Idle,
    ManeuverSelectBoost,
    ManeuverSelectFighter,
    ManeuverSelectDestination,
    AttackSelectAttacker,
    AttackSelectCard,
    AttackBeastBoost,
    AttackSelectTarget,
    AttackSelectDefenseCard,
    AttackElementaryPrediction,
    SchemeSelectCard,
    SchemeSelectTarget,
    SchemeSelectDestination,
    SchemeSelectNamedValue,
    SchemeSelectOpponentCard,
    SchemeStepLightlyTarget,
    RaveningTarget,
    RaveningDestination,
    RaveningContinue,
    ConfoundYesNo,
    ConfoundDiscardCard,
    ConfoundFogSelect,
    ConfoundFogDestination,
    CodedNotesSelectCards,
    LurkingChoice,
    LurkingFogToken,
    LurkingDestination,
    SlipAwayFogToken,
    SlipAwayDestination,
    RollingFogFogToken,
    RollingFogDestination,
    FogTokenSelect,
    FogTokenDestination,
    VanishedPlacement,
    InfoPopup,
    DraculaAbilityTarget,
    DiscardSelectCard,
    OptionalMovementDestination,
    LoadGame,
    GameOver,
};
```


9. Resource Management

ResourceManager loads and caches SFML textures and fonts. A resource is loaded only once and reused on later requests. If loading fails, an exception is generated with the requested file path.

9. مدیریت منابع گرافیکی

ResourceManager وظیفه Load و Cache کردن Texture ها و Font های SFML را دارد. هر Resource فقط یکبار Load شده و در دفعات بعدی دوباره استفاده می‌شود. اگر بارگذاری ناموفق باشد، Exception همراه با Path فایل ایجاد می‌شود.

10. Save, Load and Undo

Phase 2 adds save/load support so a match can continue from the same state. GameController provides saveGame(), loadGame() and save-slot access, while json is used for serialization. At least three save slots can be presented in the interface.

Undo is implemented with full JSON snapshots. Previous states are stored in an undo stack and the latest snapshot can be restored when the user selects Undo. The stack keeps up to 30 previous states. Throughout the game, keyboard shortcuts can be used to quickly execute commands: press Z for Undo, L for Load, and S for Save

در فاز دوم قابلیت Save/Load اضافه شده تا بازی دقیقاً از State ذخیره شده ادامه پیدا کند. GameController متدهای saveGame() ، loadGame() و دسترسی به Save Slot ها را دارد و برای Serialization از json استفاده می‌شود. در رابط برنامه امکان نمایش حداقل سه Slot ذخیره وجود دارد. سیستم Undo با Snapshot کامل JSON پیاده‌سازی شده است State های قبلی در Undo Stack ذخیره می‌شوند و با انتخاب Undo آخرین Snapshot قابل بازیابی است. این Stack حداکثر ۳۰ State قبلی را نگهداری می‌کند. در تمام طول بازی می‌توان از میانبرهای صفحه‌کلید برای اجرای سریع دستورات استفاده کرد: کلید Z برای Undo، کلید L برای Load و کلید S برای Save است.

11. Exception Handling and OOP

The Phase 2 graphical layer keeps the same modular design. Screens inherit from the common Screen base class, resources are managed in ResourceManager, and screen ownership is handled with smart pointers. Loading errors are reported through exceptions instead of being ignored.

11. ساختار فاز دوم و مدیریت خطا

لایه گرافیکی فاز دوم نیز ساختار ماژولار پروژه را حفظ می‌کند Screen های مختلف از کلاس پایه Screen ارث‌بری می‌کنند، Resource ها در ResourceManager مدیریت می‌شوند و Ownership صفحات با Smart Pointer ها انجام می‌شود. خطاهای بارگذاری نیز با Exception گزارش می‌شوند و نادیده گرفته نمی‌شوند.

12. Challenges

- **Graphical Map Design and Coordinate Alignment**

One of the main challenges was creating the graphical board and accurately matching the map image with the coordinates defined in the graphical code. Because the original image did not perfectly match the required positions, Photoshop was used to edit and adjust the board image so that the spaces aligned correctly with the coordinates used by the program.

- **Complete Game-State Restoration**

Another major challenge was restoring the complete game state for Save, Load, and Undo. All important information, including players, fighters, cards, positions, health, turns, and other game states, had to be restored consistently to prevent errors after loading or undoing an action.

- **Save-Slot Management**

Managing save slots correctly was another challenge. Each saved game had to be stored in the correct slot and later identified and loaded without overwriting or mixing different saved states.

- **Working with SFML and JSON**

Using SFML and json was an important part of Phase 2. SFML was used to develop the graphical interface, while JSON serialization was used for storing and restoring game data. Learning how to integrate these libraries correctly with the existing project required additional implementation and debugging.

- **Refactoring the Project to Use Inheritance**

The most difficult and time-consuming challenge was correcting an architectural issue from Phase 1. The first version of the project did not properly use inheritance, which made the code difficult to extend.

Refactoring the existing implementation into an inheritance-based design required significant changes to the project structure. This process also introduced several complex bugs that required extensive debugging before the project became stable again.

- **Transferring Game Logic from TUI to GUI**

Another important challenge was transferring the existing game flow from the TUI version to the graphical interface. The game rules had to remain consistent while user input and multi-step actions were adapted to an event-driven graphical environment. This required careful coordination between the existing game logic and the new GUI states.

- **Implementing Multi-Step Invisible Man Cards**

Another challenging part was implementing the Invisible Man's multi-step cards. Some card effects cannot be completed in a single action and require several consecutive player choices, such as selecting a fighter, destination, or Fog Token. Managing these steps correctly while keeping the game state synchronized with the graphical interface required careful state management and debugging.

12. چالش‌ها

- **طراحی نقشه گرافیکی و تطبیق مختصات**

یکی از چالش‌های اصلی، ایجاد نقشه گرافیکی و تطبیق دقیق تصویر نقشه با مختصات تعریف‌شده در کد گرافیکی بود. از آنجا که تصویر اولیه با موقعیت‌های موردنیاز برنامه کاملاً هماهنگ نبود، از Photoshop برای ویرایش و تنظیم تصویر نقشه استفاده شد تا خانه‌های نقشه به‌درستی با مختصات موجود در کد منطبق شوند.

- **بازیابی کامل وضعیت بازی**

یکی دیگر از چالش‌های مهم، بازیابی کامل وضعیت بازی برای قابلیت‌های Save، Load و Undo بود. تمام اطلاعات مهم از جمله بازیکنان، مبارزان، کارت‌ها، موقعیت‌ها، میزان سلامتی، نوبت‌ها و سایر وضعیت‌های بازی باید به‌صورت هماهنگ بازیابی می‌شدند تا پس از Load یا Undo خطایی در روند بازی ایجاد نشود.

• مدیریت صحیح اسلات‌های ذخیره‌سازی

مدیریت صحیح Save Slot ها نیز یکی از چالش‌های پروژه بود. هر بازی ذخیره‌شده باید در اسلات مناسب قرار می‌گرفت و در زمان Load نیز بدون تداخل یا جایگزینی اشتباه با سایر Save ها شناسایی و بازیابی می‌شد.

• کار با کتابخانه‌های SFML و JSON

استفاده از SFML و json بخش مهمی از فاز دوم پروژه بود. SFML برای پیاده‌سازی رابط گرافیکی و JSON برای Serialization برای ذخیره و بازیابی اطلاعات بازی استفاده شد. یادگیری نحوه اتصال صحیح این کتابخانه‌ها به ساختار موجود پروژه، نیازمند پیاده‌سازی و Debugging بیشتری بود.

• بازطراحی پروژه با استفاده از ارث‌بری

سخت‌ترین و زمان‌برترین چالش پروژه، اصلاح یکی از مشکلات معماری باقی‌مانده از فاز اول بود. در نسخه اولیه پروژه از Inheritance به صورت مناسب استفاده نشده بود و به همین دلیل توسعه کد دشوار شده بود. تبدیل ساختار موجود به یک طراحی مبتنی بر ارث‌بری نیازمند تغییرات گسترده در بخش‌های مختلف پروژه بود. این بازطراحی همچنین باعث ایجاد باگ‌های متعدد و پیچیده‌ای شد که رفع آن‌ها به Debugging گسترده و زمان زیادی نیاز داشت.

• انتقال صحیح منطق بازی از TUI به رابط گرافیکی

یکی دیگر از چالش‌های مهم، انتقال صحیح روند بازی از نسخه TUI به رابط گرافیکی بود. قوانین بازی باید بدون تغییر باقی می‌ماندند، در حالی که ورودی‌های کاربر و اکشن‌های چندمرحله‌ای باید با محیط گرافیکی Event-driven سازگار می‌شدند. به همین دلیل هماهنگی دقیق میان منطق قبلی بازی و State های جدید رابط گرافیکی ضروری بود.

• پیاده‌سازی کارت‌های چندمرحله‌ای مرد نامرئی

یکی دیگر از بخش‌های چالش‌برانگیز پروژه، پیاده‌سازی کارت‌های چندمرحله‌ای Invisible Man بود. افکت بعضی از کارت‌ها در یک مرحله کامل نمی‌شود و به چند انتخاب متوالی از طرف بازیکن، مانند انتخاب Fighter ، مقصد یا Fog Token نیاز دارد. مدیریت صحیح این مراحل و در عین حال هماهنگ نگه داشتن وضعیت بازی با رابط گرافیکی، نیازمند مدیریت دقیق State ها و Debugging بود.

13. Repository

The complete project code is available in the repository below; the repository itself indicates the availability of graphical and TUI versions, as well as dependencies on SFML, FTXUI, and JSON.

Unmatched Final Project — GitHub Repository :

<https://github.com/soniasaate/Unmatched-Final-project.git>

کد کامل پروژه در Repository قرار دارد؛ خود Repository نیز وجود نسخه‌های graphical و TUI و وابستگی‌های SFML، FTXUI و JSON را نشان می‌دهد.

14. Resources

- C++ Standard Library
- SFML — Graphical interface and multimedia components
- nlohmann/json — JSON serialization and game-state storage
- CMake — Project configuration and dependency management
- Git & GitHub — Version control and project repository
- Adobe Photoshop — Board image editing and coordinate alignment
- [Unmatched Project Documentation and Game Rules](#)

THANK YOU

Thank you for reading our report to the end.

And a Special thanks to our TAs for their support and guidance throughout the project.